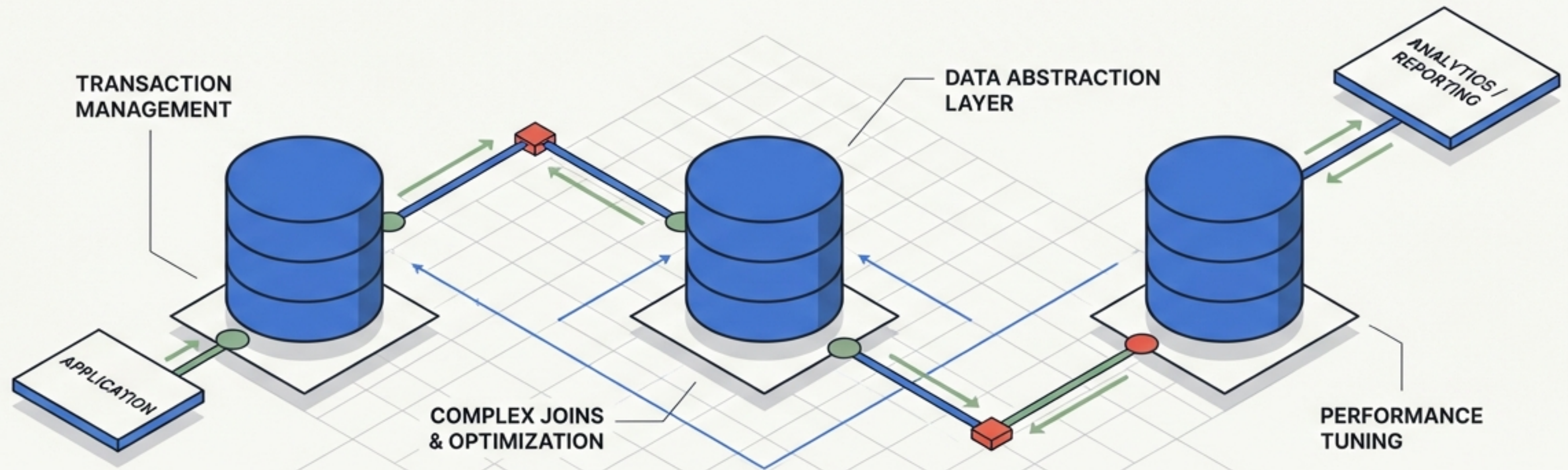
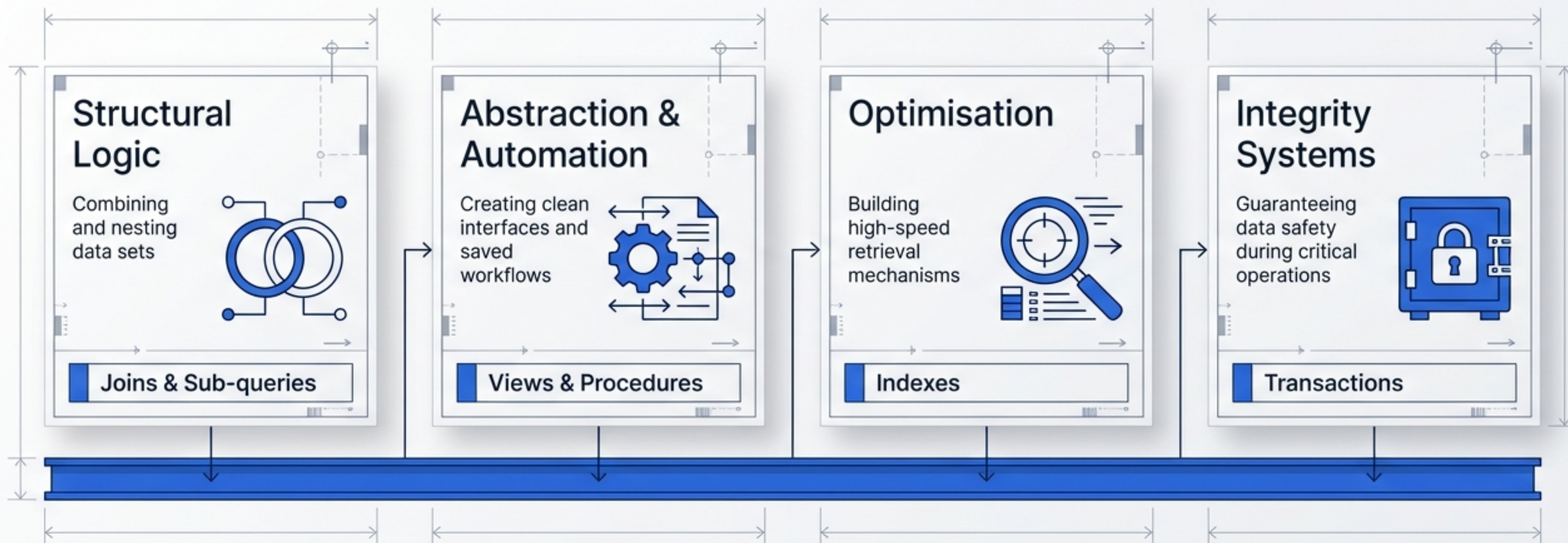


Beyond the Basics: The Advanced SQL Architecture

Mastering Transactions, Joins, Abstractions, and Performance

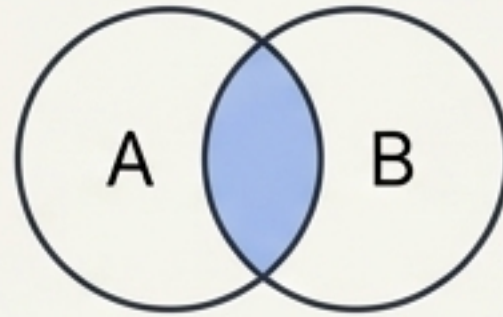


The Architect's Toolkit



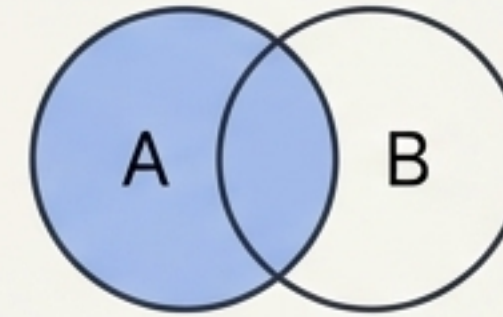
The Relational Glue: Core Combinations

Inner Join



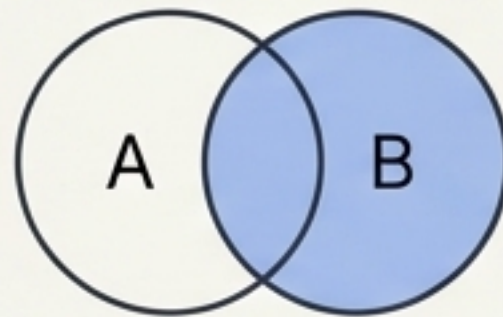
```
SELECT col FROM tableA  
INNER JOIN tableB  
ON tableA.id = tableB.id;
```

Left Join



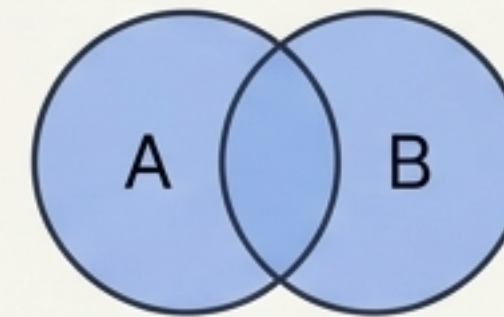
```
SELECT col FROM tableA  
LEFT JOIN tableB  
ON tableA.id = tableB.id;
```

Right Join



```
SELECT col FROM tableA  
RIGHT JOIN tableB  
ON tableA.id = tableB.id;
```

Full Outer Join

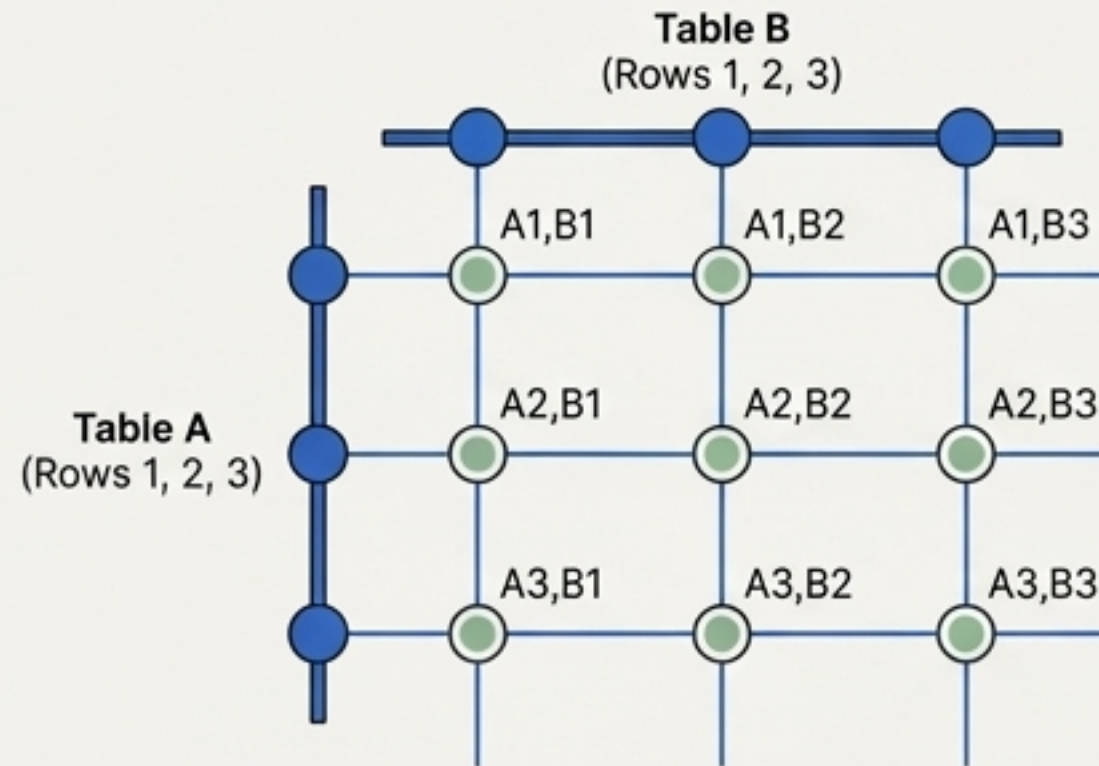


Achieved in MySQL via LEFT JOIN
combined with UNION RIGHT JOIN

Expanding the Matrix: Cross and Self Joins

Cross Join

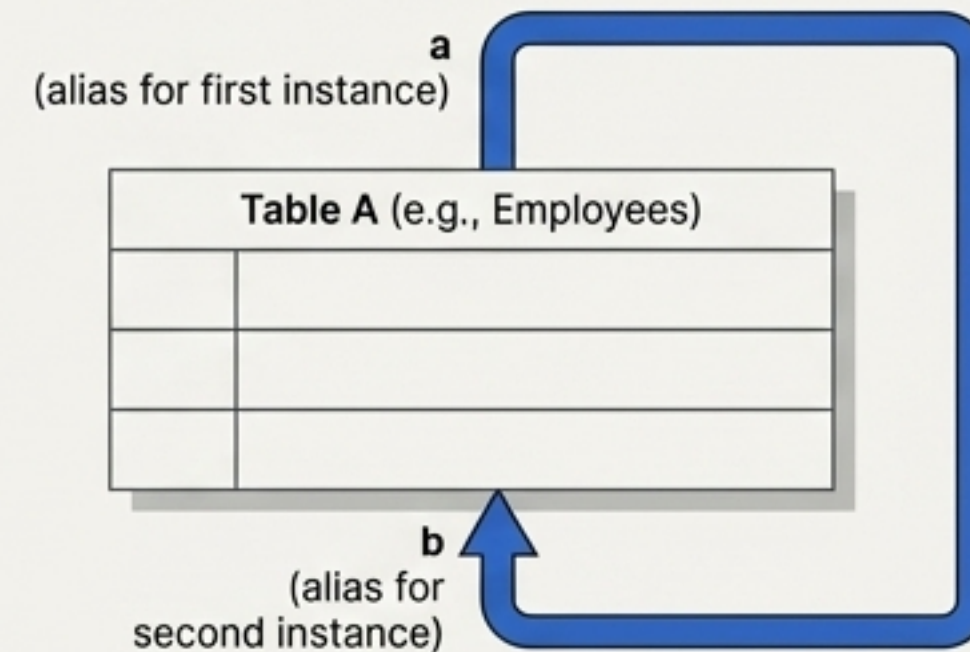
Combines every row from Table A with every row from Table B.



```
SELECT column(s)
FROM tableA
CROSS JOIN tableB ;
```

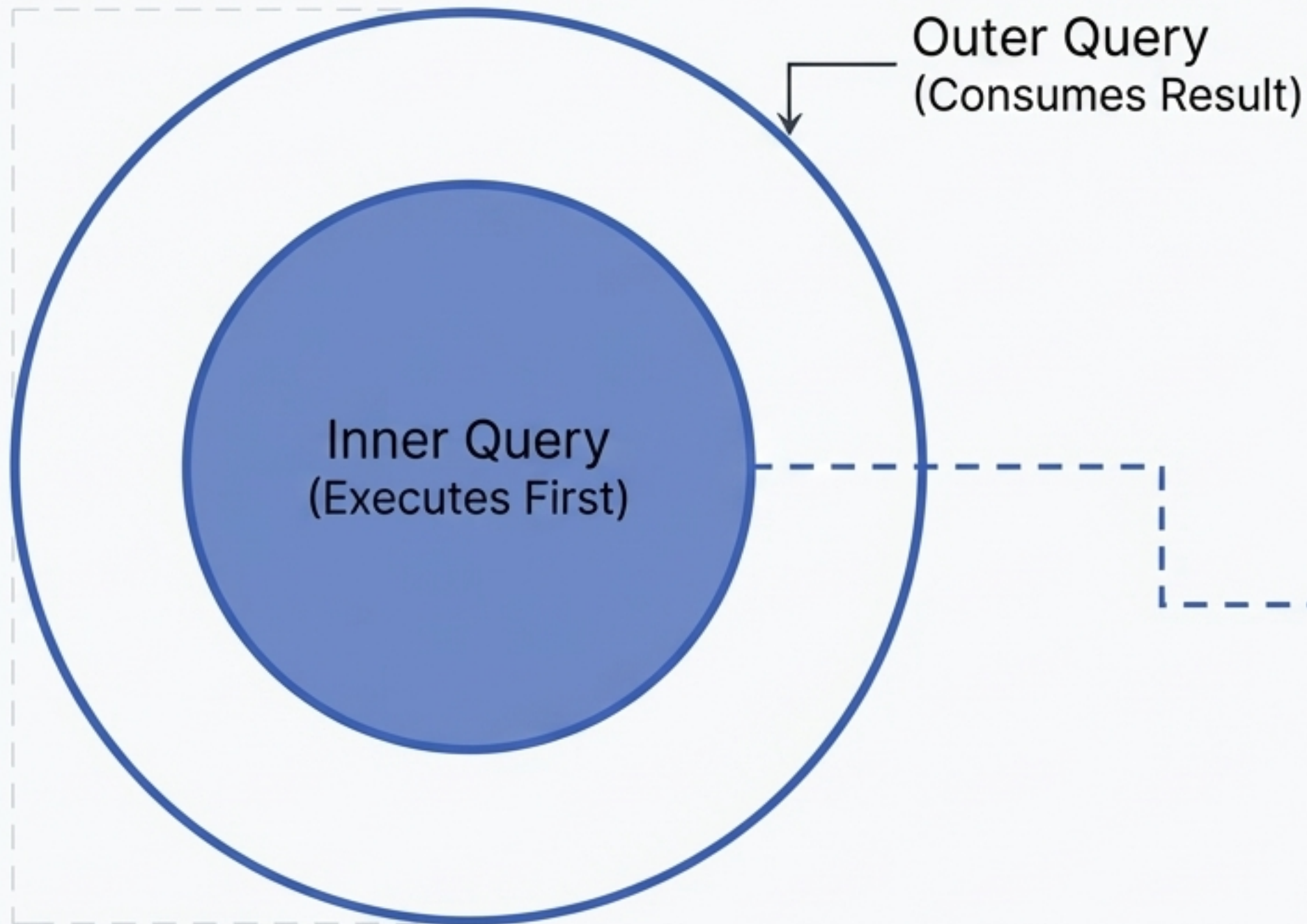
Self Join

A regular join, but the table is joined with itself. Used for hierarchical data.



```
SELECT column(s)
FROM table as a
JOIN table as b
ON a.col_name = b.col_name;
```

Sub-Queries: The Inception Technique



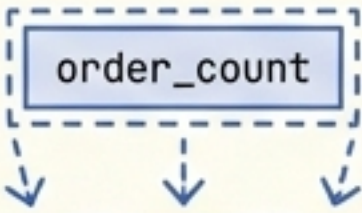
Sub-Query with WHERE clause

```
SELECT *  
FROM orders  
WHERE amount > (  
    SELECT AVG(amount)  
    FROM orders  
);
```

Sub-Queries as Virtual Environments

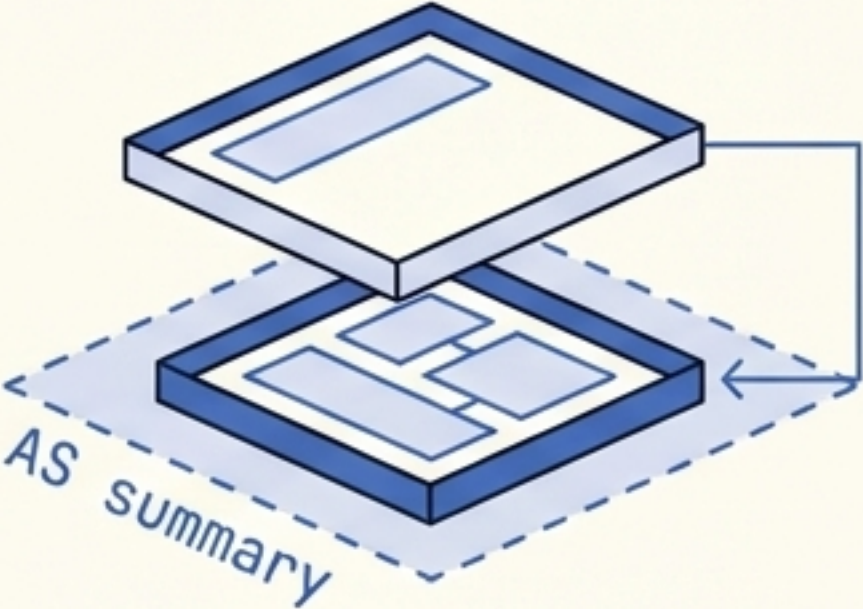
Dynamic Columns (With SELECT)

customer_id	name	...	order_count



```
SELECT name,  
       ( SELECT COUNT(*)  
         FROM orders o  
         WHERE o.customer_id = c.customer_id  
       ) as order_count  
FROM customers c;
```

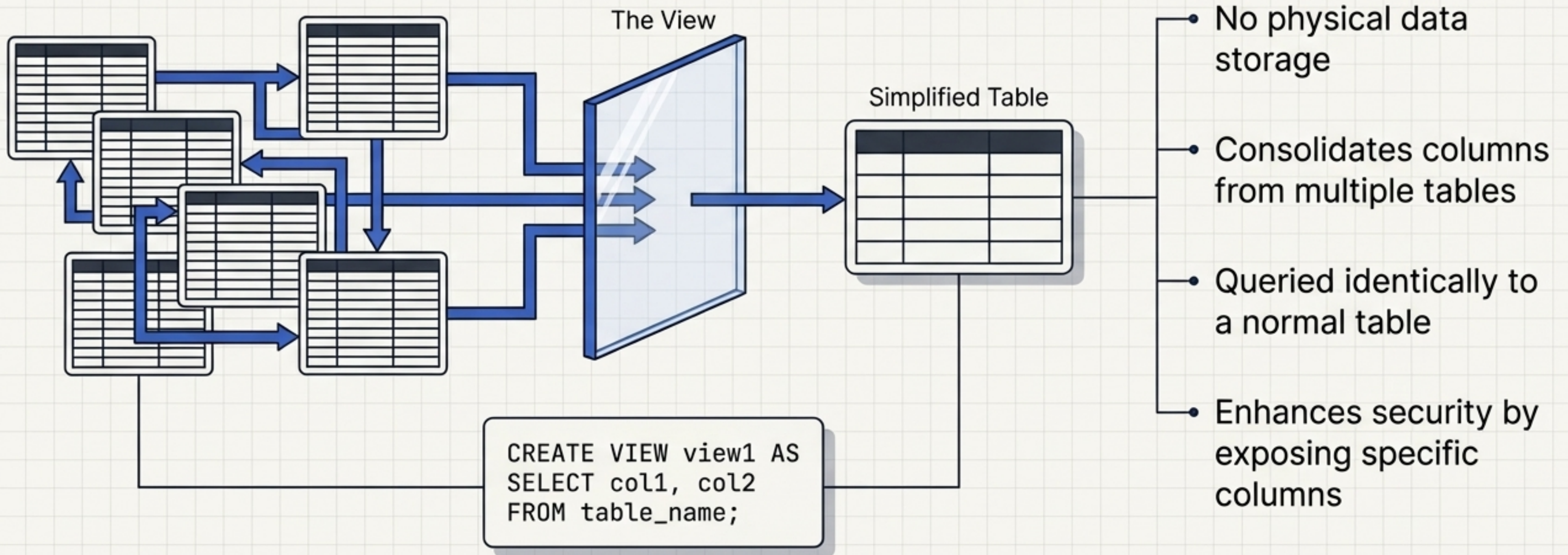
Temporary Summaries (With FROM)



```
SELECT  
  summary.customer_id,  
  summary.avg_amount  
FROM  
(  
  SELECT  
    customer_id,  
    AVG(amount) AS avg_amount  
  FROM orders  
  GROUP BY customer_id  
) AS summary;
```

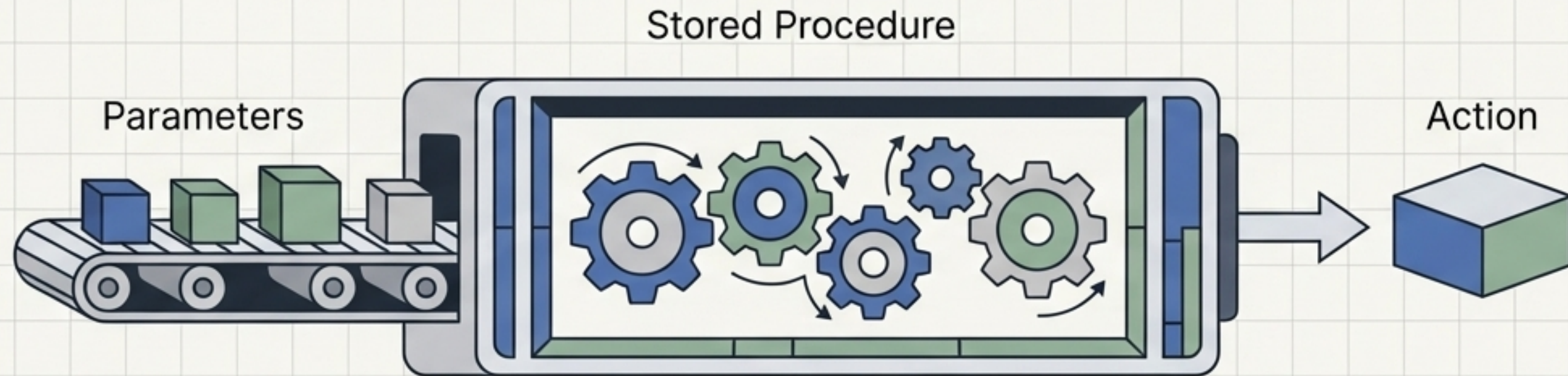
Views: The Abstraction and Security Layer

A View is a virtual table based on the result-set of an SQL statement. It shows up-to-date data without physically storing it.



Stored Procedures: The Automation Engine

Predefined set of SQL statements saved in the database, executed on demand to automate workflows.



Creation

```
CREATE PROCEDURE procedure_name (parameters)
BEGIN
    -- SQL statements
END;
```

Execution & Teardown

```
CALL procedure_name (arguments);

DROP PROCEDURE IF EXISTS procedure_name;
```

The Abstraction Matrix: Views vs. Procedures

Dimension	Views	Stored Procedures
Core Definition	A Virtual Table	A Saved Script / Action
Primary Use Case	Simplifying complex queries & securing data access	Automating business logic & multi-step data manipulation
Execution Method	Queried dynamically (SELECT * FROM view_name)	Triggered via command (CALL procedure_name)
Modifying Data	Generally read-only (complex views cannot update base tables)	Built explicitly to execute INSERT, UPDATE, and DELETE operations

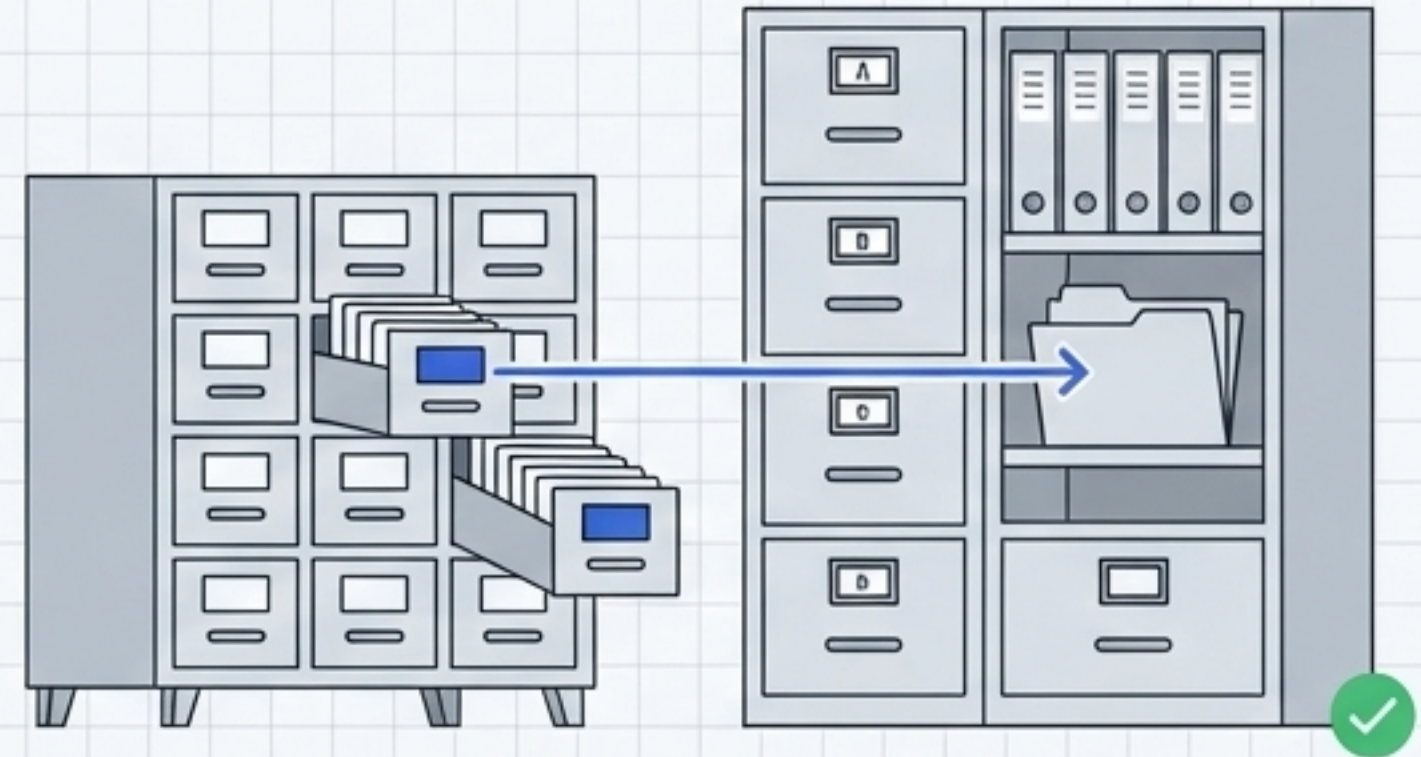
Indexes: The Performance Multiplier

Special database objects that dramatically accelerate data retrieval, preventing exhaustive full-table scans.

Full Table Scan



Index Scan



```
CREATE INDEX idx_name ON table(col);  
CREATE INDEX idx_name ON table(col1, col2);  
SHOW INDEX FROM table;  
DROP INDEX idx_name ON table;
```

Transactions: The Safety Net

A transaction is a series of operations treated as a single, indivisible whole. Designed to protect against crashes and concurrent conflicts.

Atomicity

All statements succeed, or none succeed.



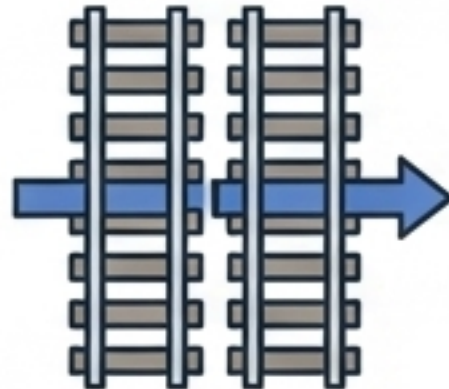
Consistency

Data moves strictly from one valid state to another valid state.



Isolation

Parallel transactions do not interfere with each other.

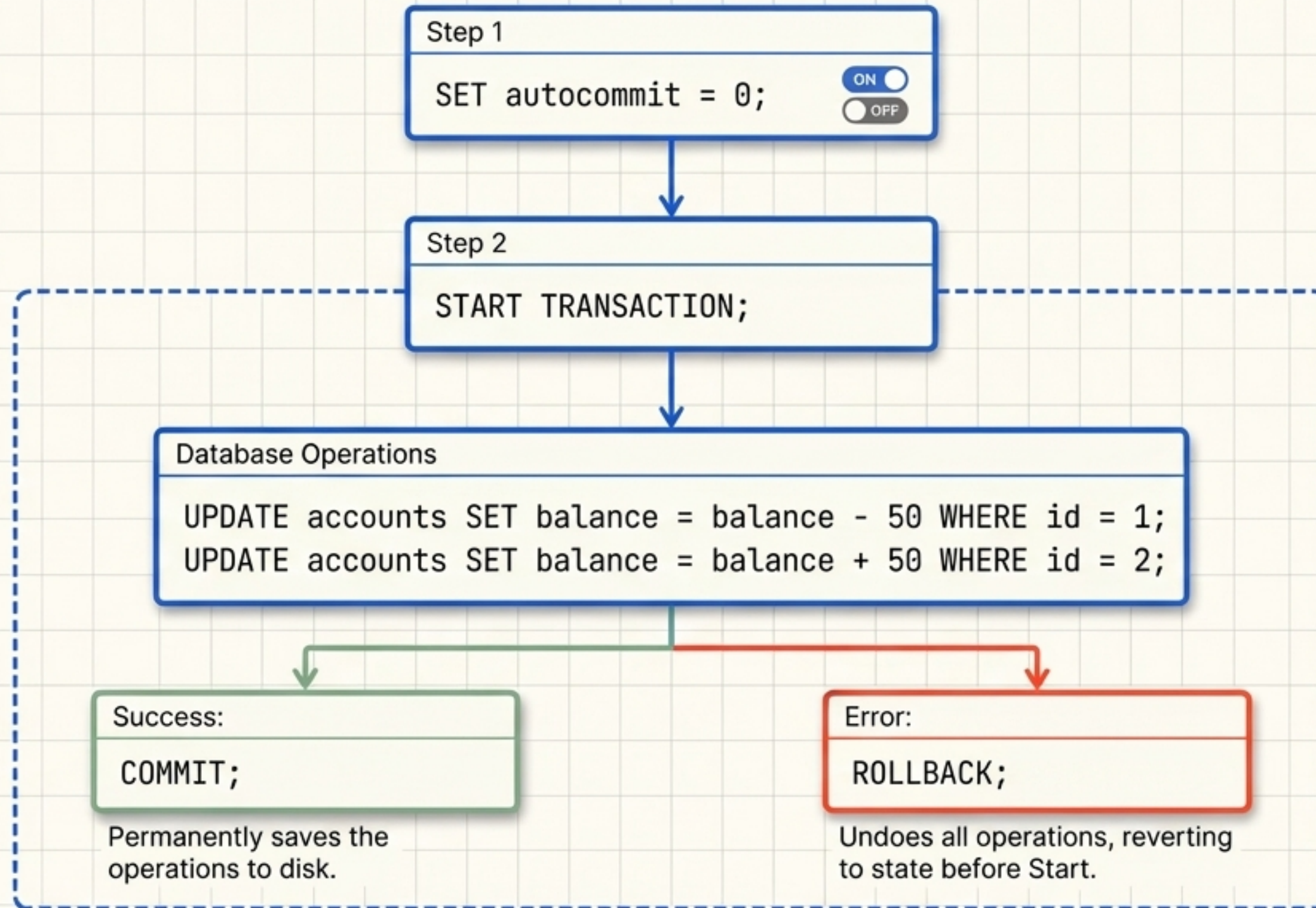


Durability

Committed data is permanently saved, surviving system failures.

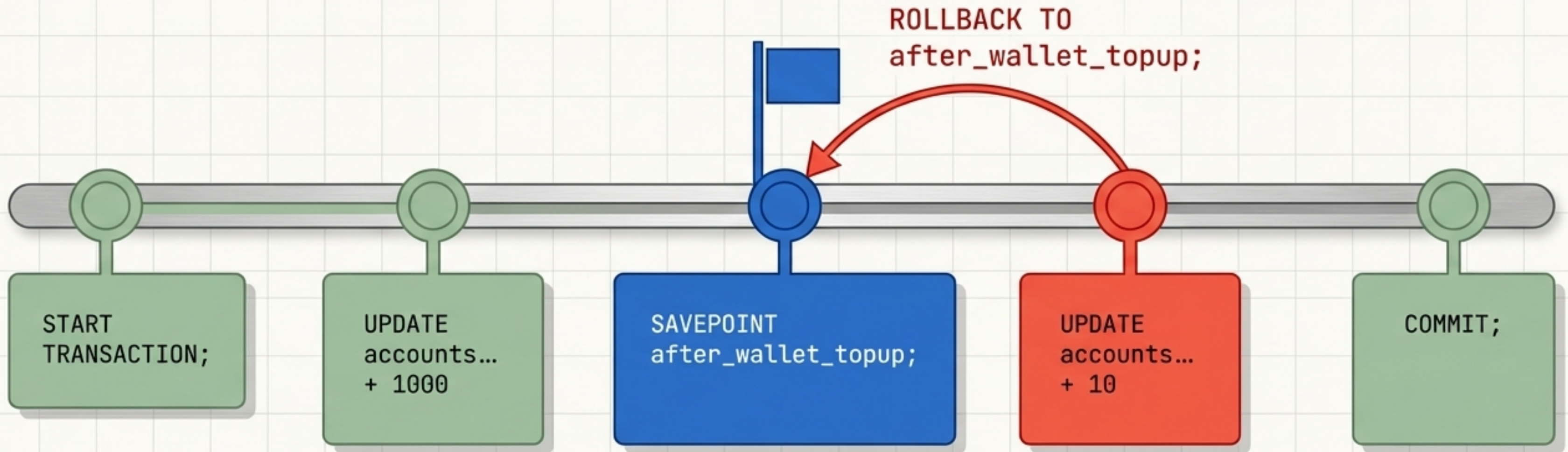


Transactions: Manual Control Flow: Commit vs. Rollback

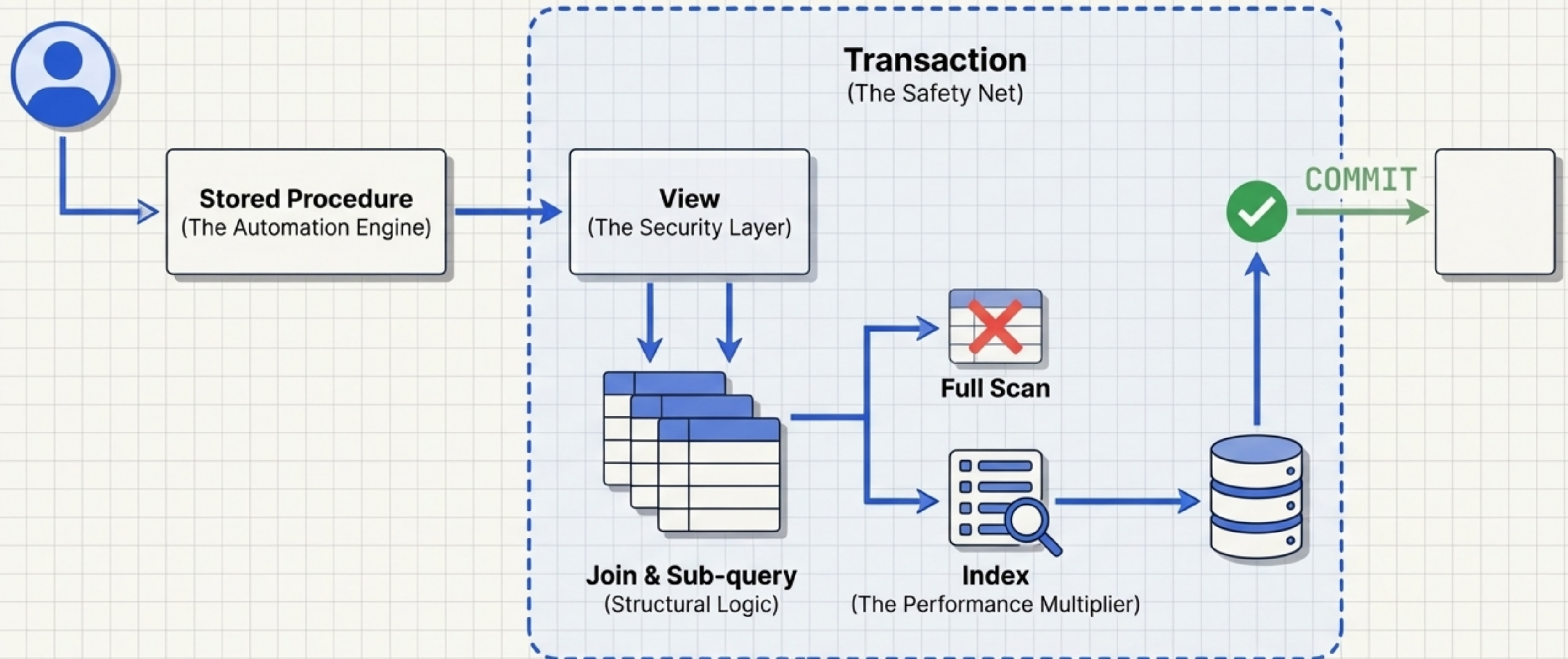


Savepoints: Granular Timeline Control

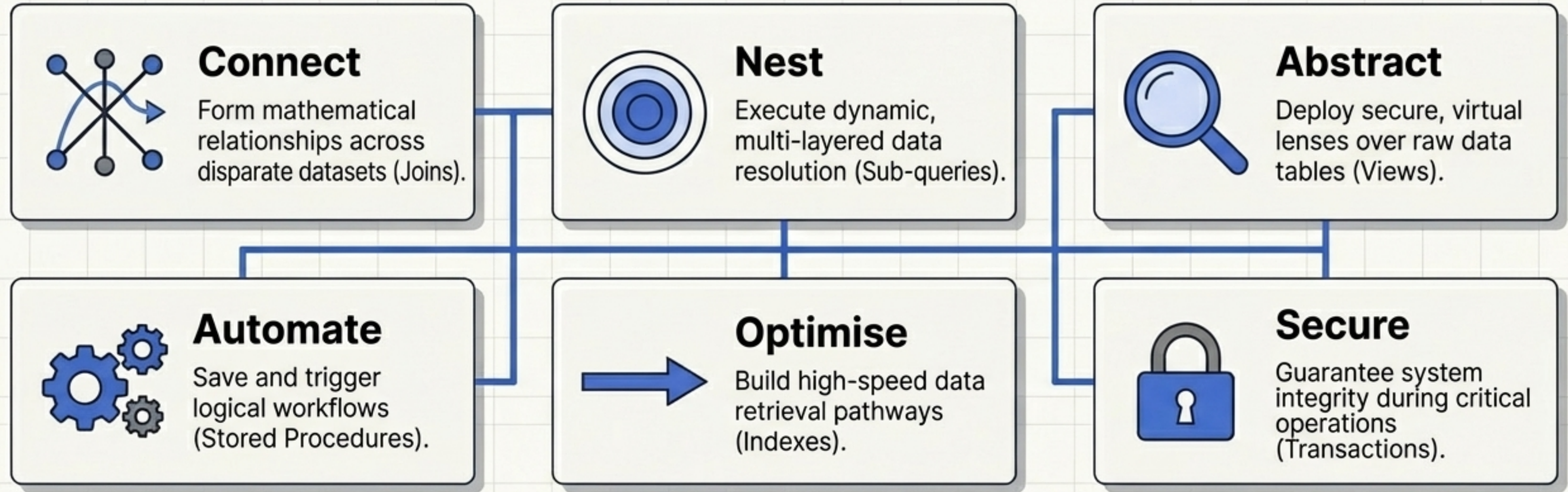
Creating safe restoration markers within a massive transaction to avoid aborting the entire process.



The Ecosystem Blueprint: Bringing It All Together



System Architecture Mastered



**The blueprint is yours.
Architect with precision.**